Weekly Student Progress Performa

Team Id	CS26084	**Week no / Date**	Week 4 10 August 2026
Project Title	SentinelCore: A Network Detection and Incident Response Platform		
Members Present	Om Rashiya (24cs084), Yash Patel (24cs082)		
Mentor	Akshita Kadam		

WEEK 4 • Repository and Full-Stack Foundation

Section	Details to be filled by the team
Planned Tasks (from last week)	Create the repository and establish a reproducible full-stack development environment.
Tasks Completed This Week	Initialized SentinelCore, created backend/frontend/helper structures, added Docker Compose services, Nginx routing, PostgreSQL and Redis, environment configuration, and application health endpoints. Established the initial React shell and FastAPI service.
Commits Made	Total: 2 commits. 6a9c10c — “Initial commit”; 96396ba — “initial project setup”.
Issues Created / Closed	Created: Coordinating service startup and database readiness. Closed: Added health checks, dependency ordering, and a single Compose workflow for the entire stack.
Milestone Status	~26% complete — on track. The full skeleton could be built and launched consistently.
Blockers / Challenges Faced	Container networking and host-port conflicts required separating internal service names from loopback-only host mappings.
Plan for Next Week	Complete container health, implement authentication/RBAC, seed an administrator, and add immutable auditing.

Research and Technical Study

Research objective. Create the repository and establish a reproducible full-stack development environment.

Study completed. Reviewed Docker Compose health semantics, reverse-proxy routing, secret injection, and reproducible environment configuration. PostgreSQL is exposed only on loopback for tools, while application traffic stays on the private Compose network.

Full-stack service topology

Service	Responsibility	Connection
Nginx	Single website/API entry	React and FastAPI
Backend	Interactive application API	PostgreSQL, Redis, helper
Worker	Ingestion and background work	Redis and PostgreSQL
Helper	Privileged network operations	Unix socket and host networking
PostgreSQL / Redis	Durable data / coordination	Private Compose network

Decision carried into the project

Initialized SentinelCore, created backend/frontend/helper structures, added Docker Compose services, Nginx routing, PostgreSQL and Redis, environment configuration, and application health endpoints. Established the initial React shell and FastAPI service.

Implementation Record and Website Evidence

Repository structure

Created distinct backend, frontend, helper, deployment, migration, and test areas so privileged and unprivileged responsibilities remained visible in the codebase.

Container topology

Added PostgreSQL, Redis, backend, worker, frontend, Nginx, helper, and Suricata-related configuration to the Compose-led development environment.

Routing and health

Nginx routes the website and API through one entry point. Service health checks and dependency ordering prevent application services from assuming the database or Redis is ready too early.

Application shell

The authenticated overview shell, sidebar, top bar, status indicators, and first dashboard cards established the common layout used by every later workspace.

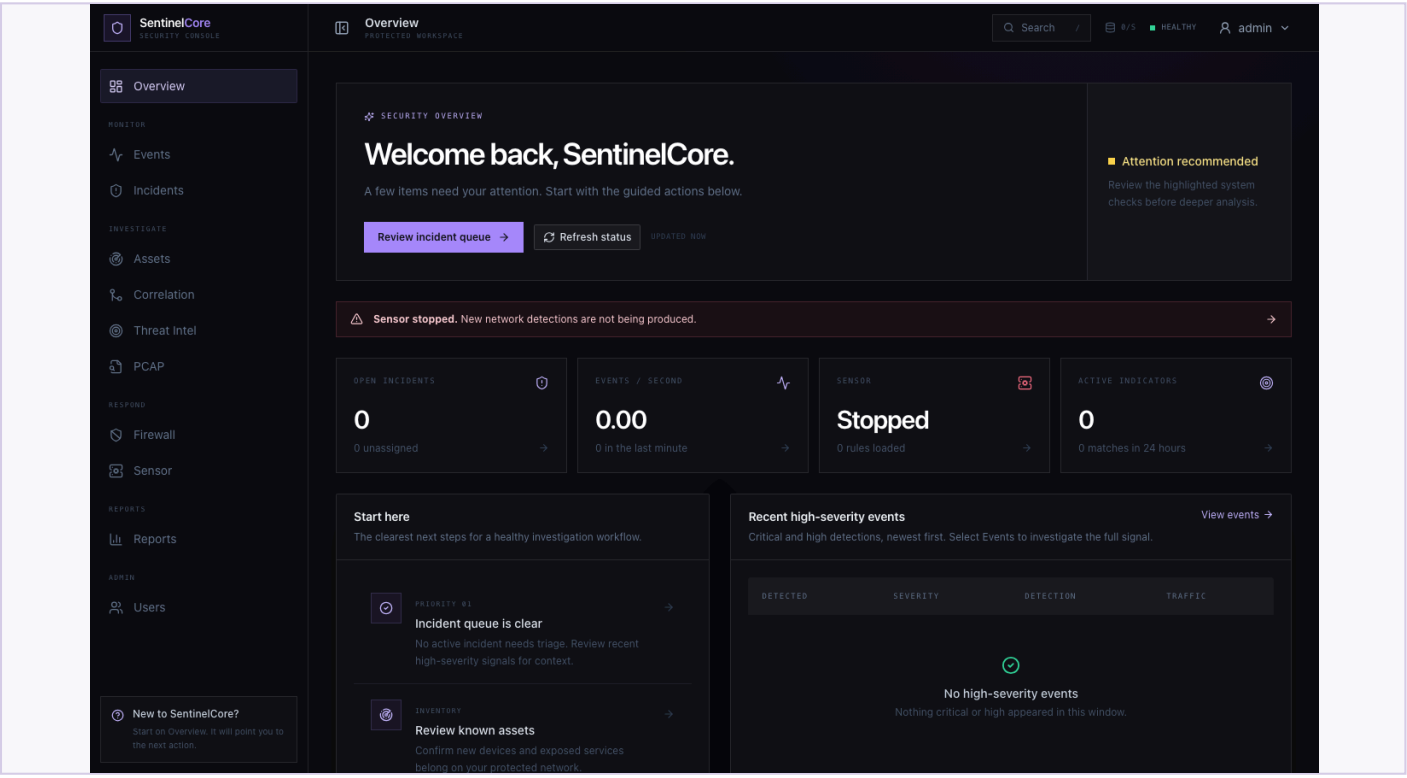


Figure W4.1 — Project-owned SentinelCore interface mapped specifically to Week 4.

Evidence interpretation. The authenticated overview shows the working application shell, role-aware sidebar, sensor warning, incident/event summary cards, and guided starting points. Its visible controls, labels, status fields, and navigation provide direct evidence for the work described on this page.

Repository and Architecture Evidence

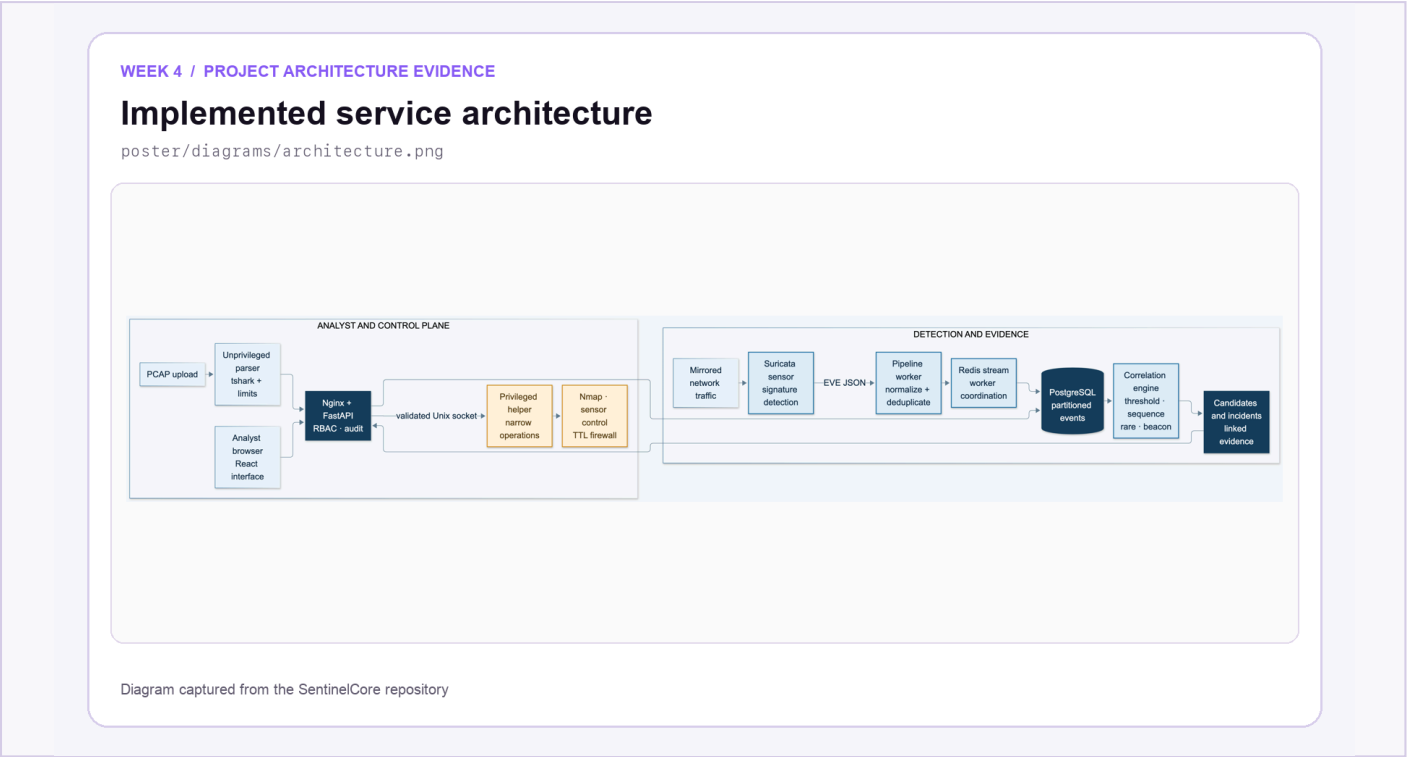


Figure W4.2 — Implemented service architecture. This is a separate internal artifact, not a copy of the website image.

Repository files used for traceability

- docker-compose.yml
- backend/deployment/nginx/nginx.conf
- frontend/src/layout/Layout.jsx
- frontend/src/pages/overview/OverviewPage.jsx

Verification

Commit hashes were taken from repository history. Visible dashboard terms and navigation labels were checked against the implemented React components.

Traceability conclusion. Implemented service architecture directly supports the Repository and Full-Stack Foundation milestone. Commit hashes were taken from repository history. Visible dashboard terms and navigation labels were checked against the implemented React components. The artifact, source paths, implementation record, and website evidence therefore describe the same weekly scope.

Weekly Closure and Faculty Assessment

Completed outcome

Initialized SentinelCore, created backend/frontend/helper structures, added Docker Compose services, Nginx routing, PostgreSQL and Redis, environment configuration, and application health endpoints. Established the initial React shell and FastAPI service.

Commit record

Total: 2 commits. 6a9c10c — “Initial commit”; 96396ba — “initial project setup”.

Issue and resolution

Created: Coordinating service startup and database readiness. Closed: Added health checks, dependency ordering, and a single Compose workflow for the entire stack.

Milestone position

~26% complete — on track. The full skeleton could be built and launched consistently.

Challenge handled

Container networking and host-port conflicts required separating internal service names from loopback-only host mappings.

Next-week hand-off

Complete container health, implement authentication/RBAC, seed an administrator, and add immutable auditing.

Repository reference

https://github.com/rashiyaom/SentinelCore

Assessed By (Faculty)	Marks Awarded (5 marks)	Faculty Remarks